Lucerne University of Applied Arts and Sciences
Business
MSc Applied Information and Data Science

W.DWL03.FS25_Data Warehouse and Data Lake Systems

Flight Delay Intelligence Platform

Author: Simão Garcia, 23-576-259, simao.garcia@stud.hslu.ch

Examiners:
- Dr. Luis Terán
- José Mancera
- Jhonny Vladimir Pincay Nieves

Luzern, May 29, 2026

Table of Contents

List of Figures

List of Tables

1. Project Idea and Use Case

1.1 Topic Introduction

Aviation delays constitute one of the most persistent operational and economic challenges in the United States air transport system. According to the Bureau of Transportation Statistics, approximately 21% of all domestic scheduled departures exceed the Federal Aviation Administration's 15-minute delay threshold, generating annual direct costs to airlines of over USD 8.3 billion (BTS, 2025). Despite this scale, the analytical tools available to aviation stakeholders remain fragmented: operations teams, meteorological analysts, and management reporting functions typically operate on separate systems with no unified layer connecting live flight telemetry, real-time weather observations, and historical performance records.

This project addresses that gap through the Flight Delay Intelligence Platform (FDIP), a fully automated, serverless data lake and data warehouse deployed on Amazon Web Services. The platform integrates multiple data sources into a unified analytical platform.

1.2 Motivation

Three interconnected drivers motivated this project; operational necessity, data engineering opportunity, and analytical research findings:

From an operational perspective, the fragmentation of aviation data systems creates preventable inefficiencies. Airport slot coordinators lack historical data to identify peak congestion windows; operations control analysts cannot correlate live weather conditions with expected delay outcomes; and strategy teams cannot benchmark carrier performance against competitors using a single consistent source of truth.

The Amazon Web Services presents a technological opportunity to build scalable, cost-effective data lake and data warehouse infrastructure that integrates diverse sources without the operational overhead of traditional database cluster management.

From an empirical perspective, research in aviation operations confirms that delay patterns exhibit strong temporal, meteorological, and carrier-specific structure that can be quantified and modelled with appropriate data (Rupp et al., 2014, p. 88).

1.3 Problem Definition and Research Questions

The core problem addressed by this project is the absence of a unified, automated data platform connecting live flight telemetry, real-time weather observations, and historical BTS performance records into a single analytical layer accessible for business intelligence queries. The overarching goal is to design, implement, and validate an automated data pipeline that delivers accurate, timely flight delay intelligence answering seven specific business intelligence questions relevant to airline operations, airport management, and passenger decision-making.

The platform is designed to investigate five research questions derived from the operational and analytical gaps identified above:

- RQ1: How do departure delays vary by hour of day, and month, and what temporal patterns emerge for slot planning purposes?
- RQ2: Which domestic routes have the highest delay rate, and what is the primary delay cause category for each?
- RQ3: How does weather visibility and ceiling at the departure airport correlate with departure delay probability and magnitude?
- RQ4: How does carrier on-time performance vary across the US domestic network?
- RQ5: What proportion of total delay minutes is attributable to carrier operations versus weather, NAS/ATC restrictions, and late inbound aircraft?

1.4 Business Intelligence Questions

The data warehouse layer answers seven specific BI questions, each mapped to a stakeholder persona and a gold-layer Athena view. Table 1 summarises the BI questions.

***Table 1**: Business Intelligence Questions*

BI#	Question	Stakeholder	Athena View
BI1	How do delays vary hourly?	Slot Coordinator	vw_airport_hourly
BI2	Which routes have the highest delay rate and primary cause?	Strategy Team	vw_route_summary
BI3	How does FAA weather category correlate with departure delays?	Operations Analyst	vw_weather_delay
BI4	Which carriers perform best on competitive routes?	Strategy Team	vw_carrier_perf
BI5	What causes delays — carrier, weather, NAS, or late aircraft?	Operations Analyst	vw_delay_causes
BI6	Which routes are consistently the most delayed?	Strategy Team	vw_route_summary
BI7	Which carrier has the best overall on-time performance?	Passengers / Analysts	vw_carrier_perf

2. Data Lake

2.1 Data Sources

The platform integrates three heterogeneous sources:

AviationStack API is a commercial REST service providing live flight status data at 100 records per daily call.

NOAA METAR API is a fully public REST service providing real-time weather observations for 20 major US airports, including the Federal Aviation Administration (FAA) flight category (VFR, MVFR, IFR, LIFR) that is central to the weather-delay correlation analysis.

BTS On-Time Performance database is a static monthly CSV publication from the Bureau of Transportation Statistics containing every US domestic scheduled flight record with FAA-standardised delay cause codes; 13 monthly files from February 2025 through February 2026 were uploaded to the bronze zone. Table 3 summarises the key characteristics and limitations of each source.

Table 2: *Data Source Overview*

Source	Type	Format	Key Variables	Limitation
AviationStack API	Dynamic (daily)	JSON	Flight status, dep/arr delays, carrier, aircraft type	Free tier: 100 records/call — partial live coverage
NOAA METAR API	Dynamic (daily)	JSON	Visibility (sm), ceiling, wind speed, flight category	20 airports only — partial weather coverage
BTS On-Time Performance	Static (monthly)	CSV	DEP_DELAY, ARR_DELAY, 5 FAA cause codes, carrier, origin, dest	13 months — Feb 2025 to Feb 2026

2.2 Architecture

The platform follows the Medallion Architecture pattern (Databricks, 2025), organising data into three progressive quality layers within Amazon S3. The Bronze layer stores raw, immutable data exactly as received from each source. The Silver layer holds cleaned, validated, and type-cast data in columnar Parquet format. The Gold layer contains pre-aggregated, analytics-ready tables optimised for business intelligence queries. Each layer is implemented as a separate S3 bucket, providing independent access control, lifecycle policies, and cost monitoring per zone. Above the storage layers, the AWS Glue Data

Catalog registers the schema of all gold tables, making them immediately queryable through Amazon Athena without manual DDL.

The system follows an event-driven orchestration pattern where Amazon EventBridge triggers Lambda functions at midnight Zurich time. Lambda writes raw data to the bronze zone. The Glue built-in scheduler then triggers the bronze-to-silver ETL job at 00:30 and the silver-to-gold ETL job at 01:00, maintaining 30-minute gaps that provide sufficient buffer for upstream jobs to complete before downstream jobs begin reading their outputs. Athena views over the gold catalog are immediately consistent following each silver-to-gold run. This fully automated nightly pipeline requires zero manual intervention once configured. Figure 1 illustrates the complete data flow from source systems to the business intelligence layer.

The platform relies on seven AWS managed services, each assigned to a specific layer of the pipeline. The entire stack is serverless and pay-per-use: no resources are running or incurring cost outside the nightly execution window. Table 3 summarises each service and its role.

Table 3: *AWS Services and Their Roles*

Layer	Service	Format	Role
Ingestion	AWS Lambda (Python 3.11)	—	Two functions poll external APIs nightly and write raw snapshots to S3 bronze
Bronze	Amazon S3	JSON + CSV	Raw immutable zone — s3://bronze-604415812723/
ETL B→S	AWS Glue 5.1 (PySpark)	—	Cleans, validates, and type-casts raw data — G.1X, 4 workers — 00:30 Zurich
Silver	Amazon S3	Parquet/Snappy	Clean structured zone — s3://silver-604415812723/
ETL S→G	AWS Glue 5.1 (PySpark)	—	Aggregates silver tables into 6 gold tables — 01:00 Zurich
Gold	Amazon S3	Parquet/Snappy	Analytics zone — s3://gold-604415812723/
Catalog	AWS Glue Data Catalog	Metadata	Schema registry — fdip_catalog — 6 tables registered by fdip-gold-crawler
Query	Amazon Athena	SQL	Serverless SQL engine — 6 views — USD 5/TB — sub-2-second response
Scheduling	EventBridge + Glue Scheduler	Cron	Nightly full automation — cron(0 22 * * ? *)

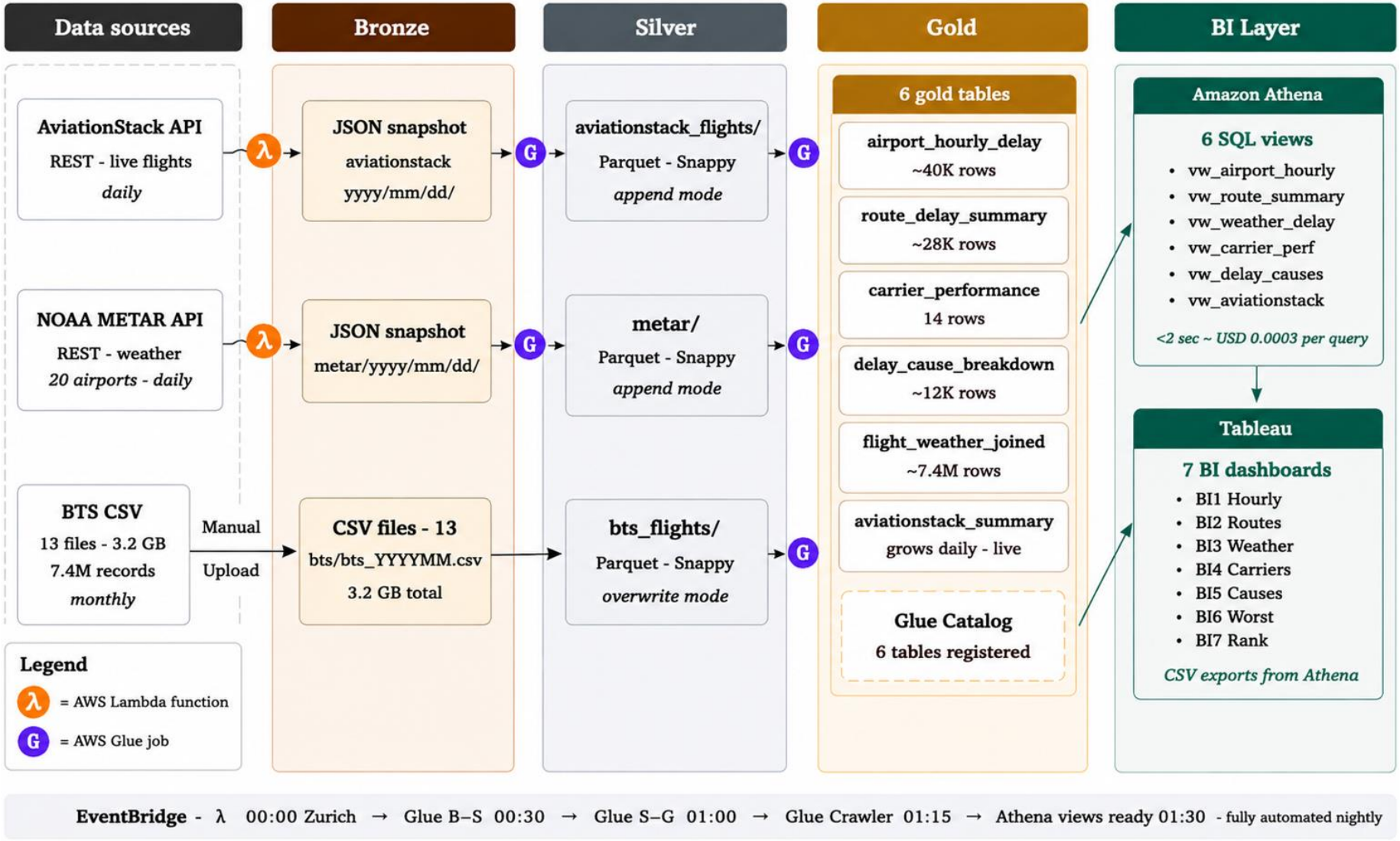

***Figure 1**: FDIP Medallion Data Lake Architecture on AWS*

3. Data Ingestion

3.1 Data Source Documentation

3.1.1 AviationStack API

AviationStack provides live flight status through a REST API at the /v1/flights endpoint, parameterised with flight_status=active and limit=100. Authentication is performed via an API key passed as a URL query parameter, stored as a Lambda environment variable. The free tier returns a single JSON page per call containing a nested data array of flight objects. No pagination logic is required at this tier.

3.1.2 NOAA METAR API

The NOAA Aviation Weather Center exposes METAR observations at https://aviationweather.gov/api/data/metar. No authentication is required. The platform requests observations for 20 major US airports in a single call using comma-separated ICAO codes. The FAA flight category field (fltCat) encodes ceiling and visibility thresholds into four categorical levels (VFR, MVFR, IFR, and LIFR) which serve as the primary weather predictor in the delay correlation analysis.

3.1.3 BTS On-Time Performance Database

The Bureau of Transportation Statistics (BTS) publishes monthly CSV files containing every US domestic scheduled flight, including departure and arrival performance and five FAA standardised delay cause columns (CarrierDelay, WeatherDelay, NASDelay, SecurityDelay, LateAircraftDelay). The 13 monthly files used in this project cover February 2025 through February 2026, totalling approximately 3.2 GB and 7,399,922 records. These files were downloaded from transtats.bts.gov and uploaded once to the bronze zone; no automated ingestion mechanism is required for this static source. Table 4 provides the complete file inventory.

Table 4: *BTS Monthly File Inventory*

File	Size	Period
bts_202502.csv	217.7 MB	February 2025
bts_202503.csv	259.0 MB	March 2025
bts_202504.csv	251.8 MB	April 2025
bts_202505.csv	261.6 MB	May 2025
bts_202506.csv	264.5 MB	June 2025
bts_202507.csv	272.9 MB	July 2025

bts_202508.csv	260.1 MB	August 2025
bts_202509.csv	242.3 MB	September 2025
bts_202510.csv	262.0 MB	October 2025
bts_202511.csv	246.1 MB	November 2025
bts_202512.csv	252.3 MB	December 2025
bts_202601.csv	233.3 MB	January 2026
bts_202602.csv	221.6 MB	February 2026
Total	~3.2 GB	Feb 2025 – Feb 2026 · 7,399,922 records

3.2 Pipeline Design

The ingestion pipeline follows a nightly batch pattern. Two Lambda functions (aviationstack-ingest and metar-ingest) are triggered simultaneously at 00:00 Zurich time by Amazon EventBridge classic rules. Classic rules attach directly as Lambda triggers and reuse the existing execution role, avoiding this restriction. Each Lambda function writes a date-partitioned JSON snapshot to the S3 bronze zone, providing idempotent writes — re-running a function on the same day overwrites only that day's file while preserving all other dates. BTS data is uploaded manually once per month when BTS publishes a new file. Figure 2 illustrates the full ingestion architecture.

The **aviationstack-ingest** polls the AviationStack REST API at the /v1/flights endpoint and extracts live flight data including flight status, departure and arrival delays, airline code, flight identifier, aircraft type, origin, destination, and scheduled departure time — up to 100 records per call in JSON format

The **metar-ingest**, polls the NOAA METAR public API and extracts weather observations for 20 major US airports, capturing station identifier, visibility in statute miles, wind speed in knots, temperature, ceiling height, observation timestamp, and FAA flight category (VFR, MVFR, IFR, or LIFR).

Each function writes a date-partitioned JSON snapshot to the S3 bronze zone, ensuring idempotent behaviour: re-running a function on the same day overwrites only that day's file while preserving all other dates.

The **BTS** data, which contains scheduled departure and arrival performance alongside the five FAA-standardised delay cause fields (CarrierDelay, WeatherDelay, NASDelay, SecurityDelay, and LateAircraftDelay), is uploaded manually once per month when the Bureau of Transportation Statistics publishes a new file. Figure 2 illustrates the full

ingestion architecture, showing the extracted fields for each source and the resulting bronze zone folder structure. Table 5 shows detail the night pipeline schedule.

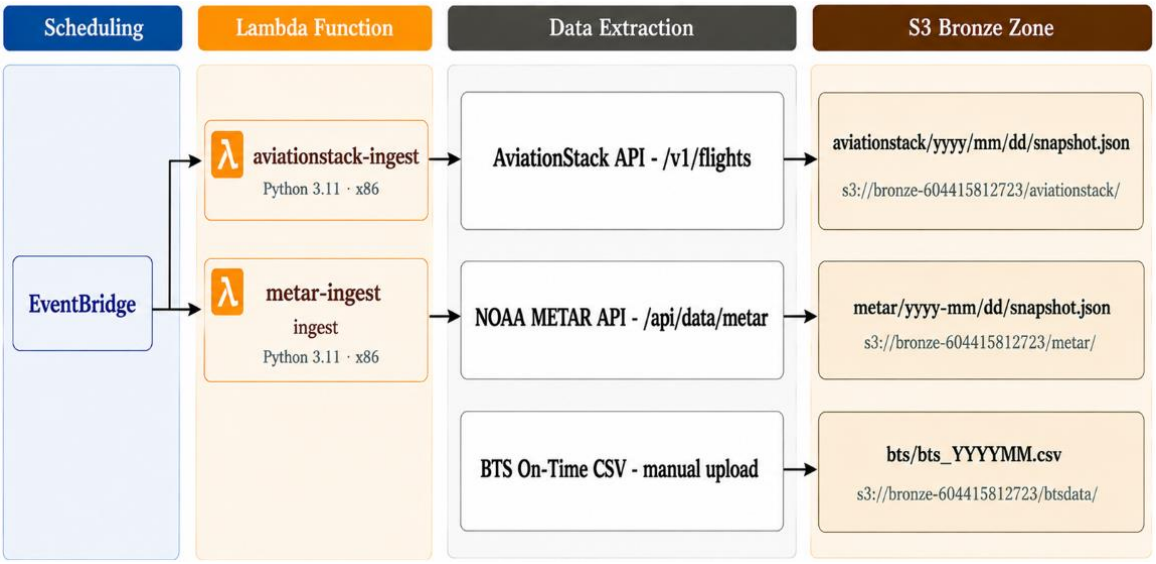

***Figure 2**: Data Ingestion Pipeline*

***Table 5**: Nightly Pipeline Schedule*

Zurich Time	Service	Action	Cron Expression
00:00	AWS Lambda (×2)	aviationstack-ingest and metar-ingest fire simultaneously	cron(0 22 * * ? *)
00:30	AWS Glue ETL	bronze-to-silver processes all 3 sources	cron(30 22 * * ? *)
01:00	AWS Glue ETL	silver-to-gold produces 6 analytics tables	cron(0 23 * * ? *)
01:15+	Glue Crawler	fdip-gold-crawler updates fdip_catalog schema	After S→G completes
01:30+	Amazon Athena	6 SQL views reflect updated gold data	Automatic

3.3 Data Capture and Handling

Schema Evolution Management: The pipeline employs schema-on-read principles enabled by Parquet's columnar format and the AWS Glue Data Catalog's schema inference capabilities. Transformation logic explicitly selects and renames the required columns by name rather than by position, ensuring resilience against source schema additions. The BTS dataset is particularly prone to field naming changes across publication versions. The bronze-to-silver job includes a diagnostic print statement that logs all detected column names to CloudWatch Logs on each run, enabling rapid identification of any future schema drift.

Error Handling and Observability: Each Lambda function and Glue job implements comprehensive try-except blocks with contextual error messages. Lambda execution logs stream to Amazon CloudWatch Logs automatically. Glue jobs write output logs to CloudWatch, where the diagnostic print statements in each try-except block confirm successful processing of each data source independently. Since each of the three data sources is processed in its own try-except block within the bronze-to-silver job, a single-source failure does not prevent the other two sources from completing successfully.

Idempotency: The BTS silver table is written in overwrite mode, ensuring that re-running the bronze-to-silver job produces a clean, consistent silver table regardless of previous executions. The AviationStack and METAR silver tables use append mode, accumulating daily snapshots incrementally. Date partitioning in the bronze zone (aviationstack/yyyy/mm/dd/snapshot.json) provides implicit versioning with one file per day, preventing duplicate records within a single day's snapshot even if Lambda is re-triggered manually.

Late-Arriving Data: METAR observations occasionally arrive with delays of up to 20 minutes due to NOAA processing latency. The silver job uses the obs_time field (the actual observation time embedded in the METAR string) rather than the Lambda invocation time as the authoritative timestamp, ensuring correct time-alignment when joining METAR to flight departure records.

4. Data Storage

4.1 Storage Service and Layers

The Amazon S3 is the storage service across the entire data lake. S3 was chosen for its effective scalability, durability, cost-effectiveness.

The storage layer follows a three-layer Medallion Architecture consisting of bronze, silver, and gold layers:

- **Bronze Zone (s3://bronze-604415812723/)**: Stores data exactly as received from source APIs (JSON) and as uploaded from BTS downloads (CSV).
- **Silver Zone (s3://silver-604415812723/)**: Stores cleaned, validated, type-cast, and deduplicated Parquet files transformed by the bronze-to-silver Glue job. The Silver layer contains: aviationstack_flights/, metar/, and bts_flights/.
- **Gold Zone (s3://gold-604415812723/)**: Stores six pre-aggregated analytics tables in Parquet format, produced by the Silver-to-Gold Glue job. It also includes an athena-results/ prefix for Athena query outputs. Figure 3 display the storage layers. Table 6 documents the complete bucket structure.

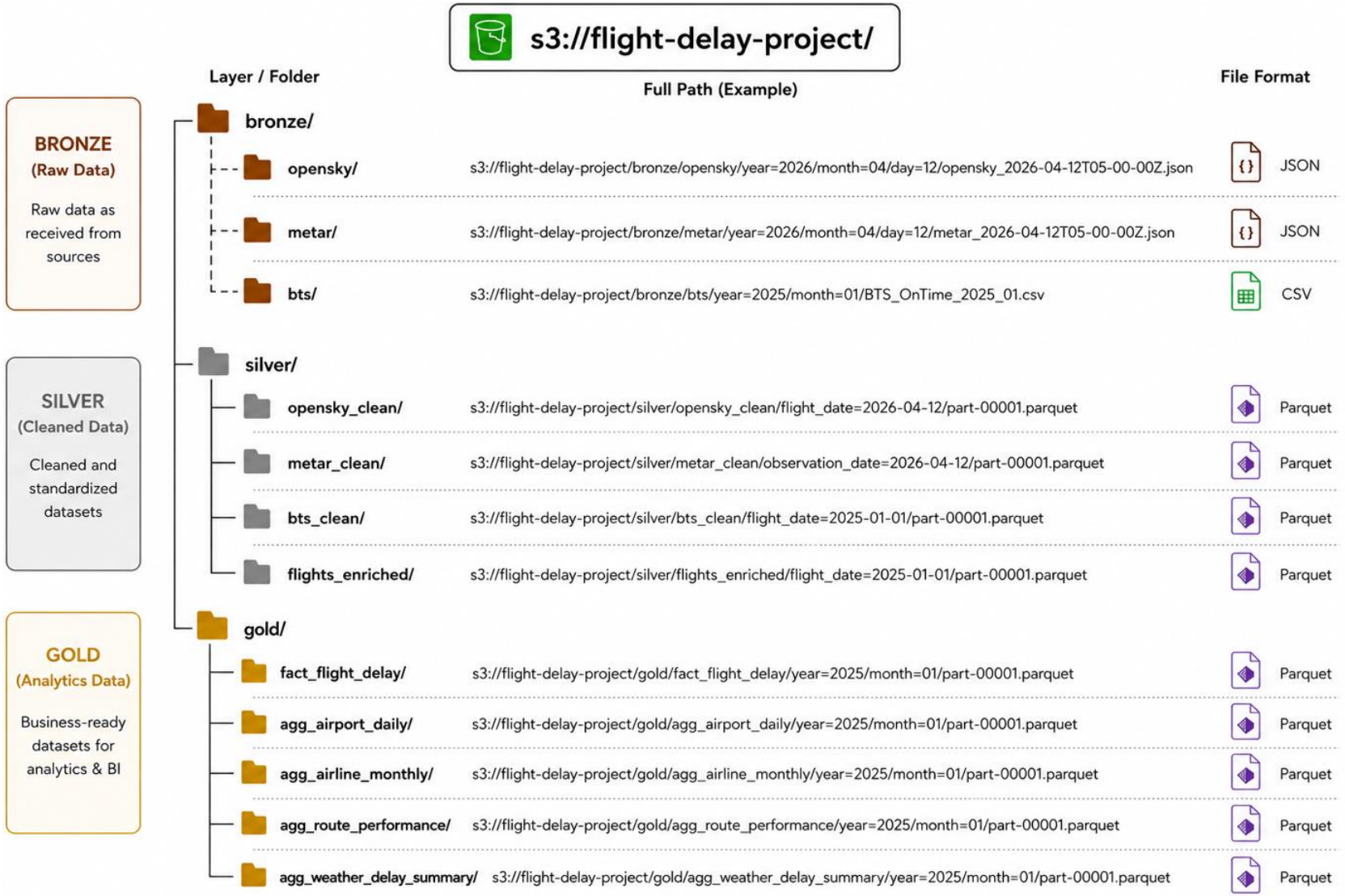

***Figure 3**: Storage layer*

Different file formats were used throughout the project depending on the processing stage and usage requirements. The objective was to preserve raw source data in the bronze layer while optimizing storage and query performance in the silver and gold layers. The table 6 summarises the decision.

Table 6: *Data Format and Optimisation by Storage Zone*

Zone	Format	Compression	Partitioning	Rationale
Bronze	JSON/CSV	None	source/year/month/day/hour	Data is stored exactly as received from the sources. aviationstack and METAR APIs data are saved as JSON. The BTS dataset is saved as CSV. This keeps the original files available for checking, debugging, and reprocessing.
Silver	Parquet	Snappy	source/year/month/day	Cleaned data is stored as Parquet because it is faster and more efficient for Athena queries than CSV or JSON. Snappy compression reduces file size while still allowing fast reads during ETL.
Gold	Parquet	Snappy	analysis_type/year/month	Gold data is already cleaned, joined, and aggregated for reporting. Parquet and partitioning help Athena read only the needed data, which improves query speed and reduces scanned data.

4.4 Database Design

Storing 7.4 million flight records across three heterogeneous sources requires a design that balances raw data preservation, analytical performance, and operational cost. The approach taken in this project keeps all data in Amazon S3 as Parquet files throughout the medallion layers, which means there is no separate database engine managing its own storage. Amazon Athena reads those files directly, translating SQL queries into distributed scans of the gold Parquet tables using the Glue Data Catalog to resolve column names, data types, and file locations. This removes the provisioning, patching, and scaling overhead of a traditional database cluster entirely.

The gold layer organises data into a fact table surrounded by four dimension tables, following the pattern established by Kimball and Ross (2013) for analytical workloads that require fast aggregation across multiple business dimensions. The central table, fact_flight_legs, records one row per scheduled flight departure with all associated delay measurements and FAA cause codes as numeric measures. Each row is linked by surrogate integer keys to dim_date for calendar-based filtering, dim_airport for origin and destination geography, dim_carrier for airline identity, and dim_delay_cause for cause classification. This structure allows Athena to answer all seven business intelligence questions through straightforward GROUP BY queries on a single table, without the multi-level joins that a more normalised design would require. Table 7 details the columns, granularity, and role of each table in the schema.

Table 7: *Star Schema*

Table	Type	Granularity	Key Columns
fact_flight_legs	Fact	One row per flight leg	FL_DATE, OP_CARRIER, Origin, Dest, DEP_DELAY, ARR_DELAY, is_delayed, is_cancelled, CARRIER_DELAY, WEATHER_DELAY, NAS_DELAY, LATE_AIRCRAFT_DELAY
dim_date	Dimension	One row per date	date_id (PK), FL_DATE, month, day_of_week, is_weekend, is_holiday
dim_airport	Dimension	One row per airport	airport_id (PK), IATA_code, ICAO_code, city, state, latitude, longitude, timezone
dim_carrier	Dimension	One row per carrier	carrier_id (PK), IATA_code, carrier_name, carrier_type
dim_delay_cause	Dimension	One row per FAA cause	cause_id (PK), cause_name, cause_code, faa_category

5. Data Transformation

5.1 Transformation Architecture

The transformation pipeline converts raw ingested data from the Bronze layer into analytics-ready tables in the silver and gold layers. The pipeline is implemented using two AWS Glue PySpark jobs that run automatically every day after the data ingestion process has completed. Figure 3 illustrates the complete two-stage transformation flow from bronze inputs through silver to the six gold output tables.

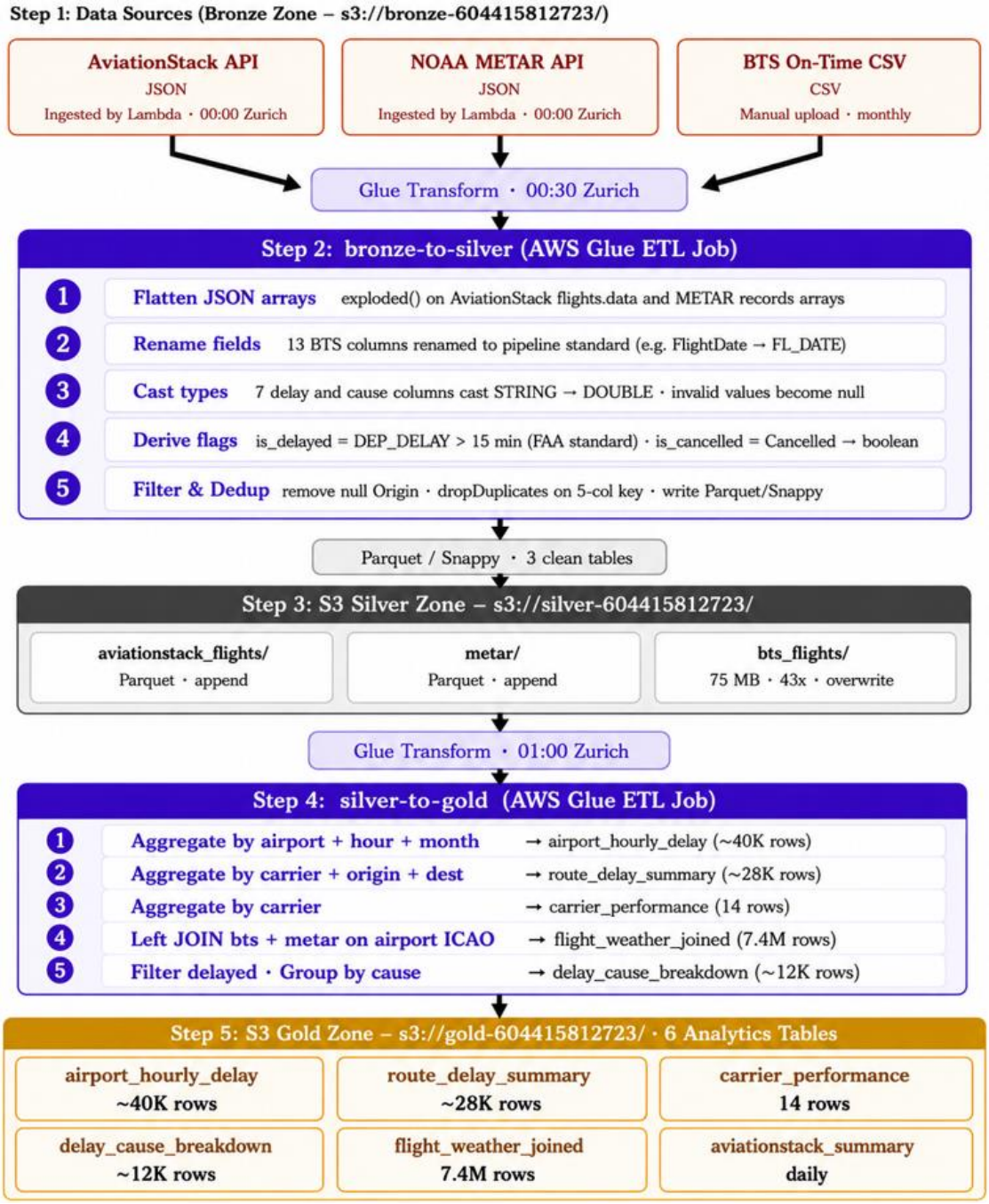

Figure 4: *Data Transformation Pipeline*

5.2 Bronze-to-Silver Transformation

The bronze-to-silver job runs daily at 00:30 Zurich time. It processes each of the three data sources in an independent fault-isolated code block, so a failure in one source does not prevent the remaining two from completing:

For AviationStack data, the nested JSON array of flight objects is flattened using the PySpark explode() function, producing one row per flight. Relevant fields are selected by name, delay columns are cast from string to double and appends the result to the silver table.

For METAR data, the JSON array is similarly flattened, and the weather attributes such as visibility, wind speed, and flight category are extracted, cast to their target types and appends the result to the silver table.

For BTS data, the transformation job reads the monthly CSV files from the bronze layer, renames columns to a consistent naming convention, converts delay-related fields to numeric data types, creates the derived fields is_delayed and is_cancelled, removes records with missing origin airports, and eliminates duplicate flight records. Table 8 summarises the key transformation steps by source.

***Table 8**: Bronze-to-Silver Transformation Steps*

Source	Operations
AviationStack	Explode nested JSON array select and rename fields cast delay columns to DOUBLE filter null origin dropDuplicates append silver bucket
METAR	Explode records array · extract icaoId, visibility, fltCat, temp, wind · cast to DOUBLE filter null airport append to silver bucket
BTS	Rename 13 fields cast 7 delay/cause columns to DOUBLE derive is_delayed (>15 min) and is_cancelled filter null Origin dropDuplicates on 5-column key overwrite to silver bucket

5.3 Silver-to-Gold Transformation

The silver-to-gold job runs daily at 01:00 Zurich time. It reads all three clean silver Parquet tables and produces six pre-aggregated gold tables used for reporting and business intelligence in overwrite mode. Table 9 maps each gold table to its source, aggregation logic, and BI question.

Table 9: *Gold Tables — Source, Aggregation, and BI Mapping*

Gold Table	Source	BI#	Key Aggregation
airport_hourly_delay	bts_flights	BI1	GROUP BY airport + dep_hour + month AVG delay · delay_rate
route_delay_summary	bts_flights	BI2, BI6	GROUP BY carrier + origin + dest AVG delay, delay_rate, AVG per cause filter ≥10 flights
carrier_performance	bts_flights	BI4, BI7	GROUP BY carrier AVG dep/arr delay delay_rate · total_cancelled
delay_cause_breakdown	bts_flights	BI5	Filter is_delayed=True · GROUP BY airport + carrier · AVG 4 causes · filter ≥5 delayed
flight_weather_joined	bts_flights + metar	BI3	LEFT JOIN bts on metar.airport_icao = bts.Origin · all flights + weather enrichment
aviationstack_summary	aviationstack_flights	Live	GROUP BY carrier + origin + dest · COUNT, AVG delays · filter ≥2 flights

5.4 Data Quality and Validation

Data quality is enforced at each layer of the pipeline through four distinct mechanisms rather than as a single downstream check.

Completeness monitoring: The completeness of each ingestion run is verified by the Lambda functions, which log the number of records returned by each API call to Amazon CloudWatch Logs on every execution, making it straightforward to detect a run that returned zero records due to an API outage or rate limit. At the silver layer, record counts before and after deduplication are logged on every run, providing a consistent and comparable measure of data volume across executions. At the gold layer, the row count of each output table serves as an implicit completeness indicator. A

route delay summary with significantly fewer rows than the previous run signals that upstream data was incomplete or a filter threshold produced an unexpectedly aggressive exclusion.

Consistency verification: The bronze-to-silver job logs all column names detected in the BTS source files to CloudWatch on every run, providing an early warning mechanism for schema drift. If a future BTS publication renames or removes a column, the discrepancy appears immediately in the logs without causing a silent data loss downstream. The is_delayed flag applies the FAA official 15-minute threshold consistently across the entire dataset, ensuring that the definition of a delayed flight is identical in every gold table and every dashboard query rather than being defined independently in each place.

Validity enforcement: Data type casting at the silver layer converts all delay and cause columns from string to double, with invalid or non-numeric values becoming null rather than causing job failures. This ensures that only parseable, typed values reach the aggregation stage. At the gold layer, minimum record thresholds are applied to avoid misleading averages: the route delay summary requires at least ten flights per route and the delay cause breakdown requires at least five delayed flights per airport-carrier combination before including a row in the output. These thresholds prevent statistically meaningless averages, such as a single severely delayed flight on a rarely operated route, from appearing in dashboard results and misleading analysts. The flight_weather_joined table applies a left join, intentionally retaining all 7.4 million BTS flight records regardless of whether a matching METAR observation exists for the departure airport, ensuring that completeness of the flight dataset is never sacrificed for weather enrichment coverage. All six gold tables are written in full overwrite mode, guaranteeing that each nightly run produces a complete and internally consistent snapshot rather than a partial append that could mix stale and current data.

Monitoring and Alerting: While the platform does not implement real-time alerting, all Lambda executions and Glue job runs write structured logs to Amazon CloudWatch Logs, enabling post-hoc analysis of any unexpected behaviour. A Lambda execution returning zero records, a Glue job completing in significantly less time than usual, or a gold table with an unexpectedly low row count are all detectable through CloudWatch without any additional instrumentation. This lightweight monitoring approach is appropriate for a nightly batch pipeline where a one-day data lag is acceptable and the primary risk is silent failure rather than real-time data corruption. Table 10 summarises the quality controls at each layer.

Table 10: *Data Quality Controls by Layer*

Layer	Control	Implementation	Purpose
Bronze	Immutability	Write-once, never modified	Full replay capability; complete audit trail
Silver	Null filtering	filter(Origin.isNotNull())	Removes structurally invalid records
Silver	Deduplication	dropDuplicates on 5-column composite key	Eliminates exact duplicate flight records
Silver	Type casting	cast('double') — invalid strings become null	Prevents type errors in downstream aggregations
Silver	Schema logging	Column names logged to CloudWatch on every run	Early detection of source schema changes
Gold	Statistical gates	≥10 flights/route · ≥5 delayed/airport-carrier	Prevents statistically meaningless aggregated averages

5.5 Scalability and Reusability

Both Glue jobs are fully idempotent and stateless. Re-running either job at any time produces a deterministic output regardless of how many previous runs have occurred. The fault-isolated source blocks in the bronze-to-silver job allow the three sources to be modified or extended independently. Adding additional months of BTS data requires no code changes. The job reads the bronze BTS path using a wildcard that automatically picks up any new files. AWS Glue automatically adjusts Spark resource allocation based on data volume, and the PySpark code supports this by following distributed computing best practices: avoiding collect() operations that would move large datasets to the driver node, and using built-in DataFrame transformations rather than row-level Python functions that cannot be parallelised across workers.

6. Data Warehouse

6.1 Business Goals and Objectives

The data warehouse layer translates the raw analytical potential of the gold zone into structured, queryable insights for three stakeholder groups:

1. Airport Slot Coordinators require hourly and monthly delay heatmaps to identify peak congestion windows and adjust gate and slot allocations proactively.
2. Airline Strategy and Revenue Management Teams require route-level delay rankings, carrier benchmarking, and delay cause attribution to evaluate competitive positioning and prioritise operational investment.
3. Operations Analysts require weather-delay correlation data to build proactive passenger rebooking protocols when adverse conditions are forecast at departure airports.

The design addresses three specific objectives:

- Establishing a single source of truth that reconciles all three data sources and ensures consistent delay metrics across all analytical use cases;
- Enabling sub-two-second query response on the pre-aggregated gold Parquet tables;
- Providing a documented SQL interface for both guided dashboard reporting and ad-hoc analysis by technical users.

The required data granularity spans two levels:

1. Flight-level records for flexible filtering and correlation, and pre-aggregated tables at route, carrier, airport-hour;
2. Airport-carrier level for fast interactive queries, supporting decisions across operational, tactical, and strategic time horizons simultaneously.

6.2 Data Modelling Approach

The gold layer adopts a star schema dimensional model, placing query efficiency and intuitive data exploration at the centre of the design. This choice deliberately accepts a degree of storage duplication relative to a normalised model, but the trade-off is clearly favourable given that all four dimension tables are small and the workload is entirely read-oriented. The denormalised structure optimises Athena read performance on columnar Parquet storage and enables straightforward SQL queries without multi-level joins.

The model is built around a single central fact table connected to four dimension tables. The central fact table, **fact_flight_legs**, contains one row per scheduled domestic flight leg with all delay measurements and the four FAA cause attribution columns as measures, connected to all four dimensions through foreign key relationships.

The four supporting dimensions each serve a distinct analytical role:

1. **dim_date** provides a date spine with derived calendar attributes (month, day of week, weekend flag, and holiday flag) enabling temporal slicing across all fact queries.
2. **dim_airport** covers approximately 500 US airports and plays a dual role, serving as the reference for both the origin and destination of every flight.
3. **dim_carrier** holds one row per carrier for the 14 US domestic airlines in the BTS dataset.
4. **dim_delay_cause** is a six-row lookup table defining the FAA-standardised delay cause categories used consistently across all aggregations and dashboards.

Surrogate integer keys are used for all dimension tables, decoupling the warehouse model from source naming conventions and ensuring efficient join performance on integer columns. The dim_airport table plays a dual role in the schema, serving as the reference for both the origin and destination of each flight through two separate foreign key relationships on the fact table. Figure 4 presents the schema in relational table format, showing all primary keys, foreign keys, column names, and data types.

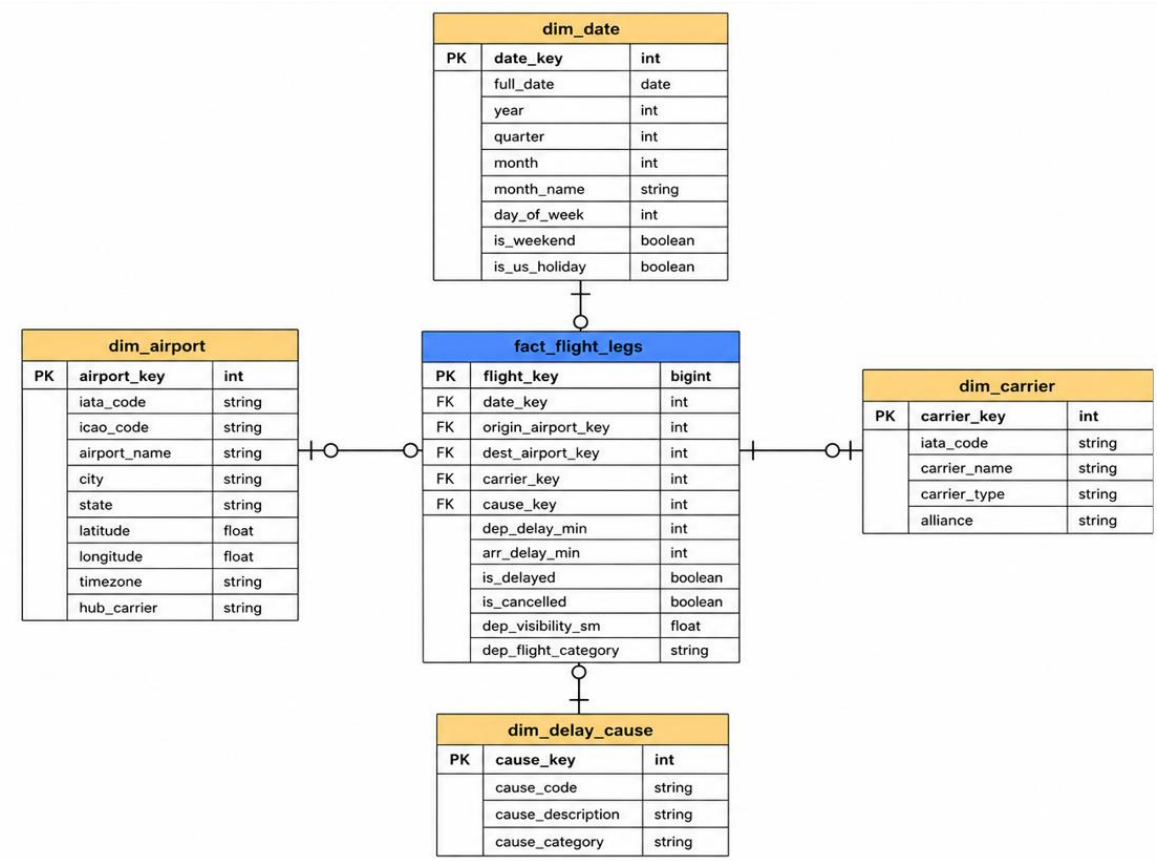

***Figure 5**: FDIP Star Schema*

6.3 Architecture

The data warehouse is implemented as a serverless lakehouse using Amazon Athena as the query engine and the AWS Glue Data Catalog as the schema metastore. Athena is natively integrated with the Glue Data Catalog, requires no cluster provisioning, and delivers sub-two-second results on the pre-aggregated gold Parquet tables. Table 11 documents the decision rationale.

This architecture offers four practical advantages for the platform's workload pattern. Cost efficiency is achieved through pay-per-query pricing with no idle infrastructure costs between nightly pipeline runs. Elastic scalability means concurrent queries and data volume growth are handled automatically without capacity planning. Simplified operations remove the need for cluster provisioning, patching, or performance tuning entirely. Finally, data locality ensures that data never moves between storage and compute layers, since Athena queries the gold Parquet files directly in S3.

6.4 Data Preparation and Loading Strategy

The ETL process feeding the warehouse is the silver-to-gold Glue job described in Chapter 5. The extraction phase reads all three silver Parquet tables using Spark distributed reads across four G.1X workers. The transformation phase applies the groupBy aggregations, LEFT JOIN enrichment, and minimum record thresholds. The loading phase writes all six gold tables as Parquet files to the gold S3 bucket in full overwrite mode. Incremental loading was evaluated and rejected because the BTS dataset is subject to retroactive corrections by the Bureau of Transportation Statistics: historical months can be updated after initial publication, and an incremental strategy would miss such corrections without implementing complex change-detection logic. Full overwrite avoids this at negligible extra compute cost, completing in approximately 12 minutes.

6.5 Athena SQL Views

Six SQL views are defined in Athena (one per gold table) to standardise column names, apply baseline quality filters, and provide clean documented query interfaces for the visualisation layer. All views use CREATE OR REPLACE VIEW for idempotent re-execution. The full view definitions are provided in Appendix A. Table 11 maps each view to its gold table, BI questions served, approximate row count, and query performance.

Table 11: *Athena Views*

View	Gold Table	BI#	Approx. Rows	Query Time
vw_airport_hourly	airport_hourly_delay	BI1	~40K	<1 sec
vw_route_summary	route_delay_summary	BI2, BI6	~28K	<1 sec
vw_weather_delay	flight_weather_joined	BI3	7.4M	~2 sec
vw_carrier_perf	carrier_performance	BI4, BI7	14	<0.5 sec
vw_delay_causes	delay_cause_breakdown	BI5	~12K	<1 sec
vw_aviationstack	aviationstack_summary	Live	Grows daily	<1 sec

7. Data Visualisation

7.1 Visualisation Purpose

The visualisation layer translates gold-layer analytics into actionable insights for the three stakeholder groups identified in Section 6.1. The platform's diagnostic analytics approach leads each dashboard with the key finding and allows users to filter by carrier, airport, month, and flight category to interrogate supporting detail. Tableau to perform data visualization. The export queries used to generate these files are provided in Appendix B.

7.2 Business Intelligence Findings

BI1: Temporal Delay Patterns by Hour

Analysis of 7.4 million flights reveals a pronounced diurnal delay pattern. Flights departing between 06:00 and 07:00 time experience the lowest average delay of approximately 0–20 minutes, because aircraft begin the operating day without accumulated disruption. As the day progresses, flights between 10am and 6pm show a moderate and stable delay level of approximately 35 to 40 minutes, reflecting background network congestion during peak hours. From 6pm onward delays escalate sharply, reaching nearly 200 minutes by 11pm as accumulated disruptions propagate through the network via the late aircraft cascade effect.

Therefore, early morning flights carry the lowest delay exposure, afternoon flights a moderate and predictable level, and evening departures after 6pm carry a substantially elevated risk of delays measured in hours rather than minutes.

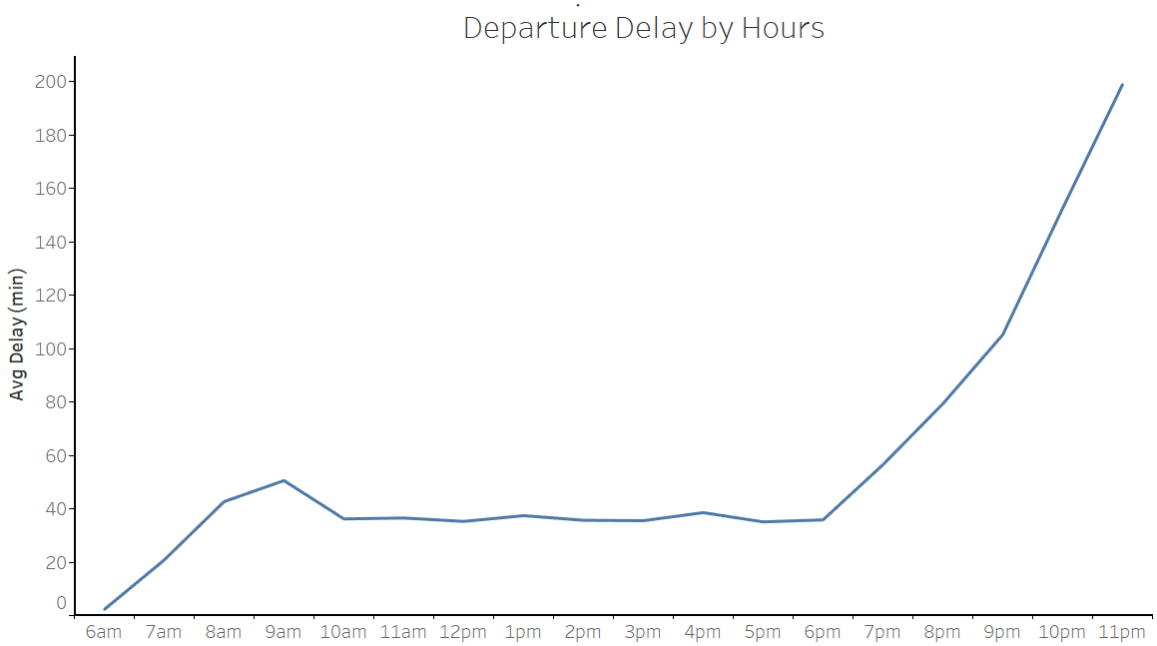

Figure 6: *Temporal Delay Patterns by Hour*

BI2: Worst Routes by Delay Rate

Nantucket Memorial to Chicago O'Hare (ACK→ORD, SkyWest/OO) is the most delayed route in the network, with passengers waiting on average 93 minutes past their scheduled departure. Anchorage (ANC) routes appear three times in the top ten, as serving remote Alaskan destinations leaves little room to recover when something goes wrong. SkyWest (OO) and Frontier Airlines (F9) each appear twice: both airlines operate with very few daily flights per route, meaning that if one flight arrives late, the next departure on that same aircraft is automatically late as well with no way to catch up.

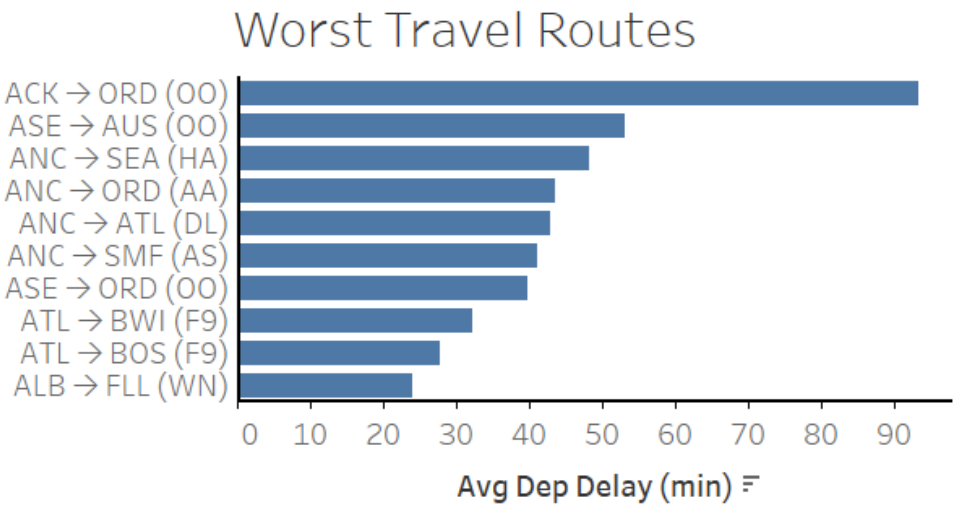

Figure 7: *Top 10 routes by delay rate*

BI3: Weather Category and Departure Delay Correlation

The analysis examines departure delays across the three FAA flight categories present in the dataset: VFR (Visual Flight Rules), MVFR (Marginal Visual Flight Rules), and IFR (Instrument Flight Rules). IFR conditions produce the highest average departure delay at 15.5 minutes with a 23.1% delay rate. VFR conditions show 14.8 minutes and 22.7% delay rate. MVFR conditions, where visibility and ceiling are marginal but still permit visual flight, show 13.2 minutes average delay and a 21.5% delay rate. While the differences between categories are modest, the direction is consistent with expectations: deteriorating visibility and ceiling height correlate with increased departure delays. Table 12 provides the full breakdown by FAA flight category.

Table 12: *Departure Delay by FAA Flight Category*

Category	Conditions	Avg Delay	Delay Rate
IFR	Ceiling 500–1,000 ft · visibility 1–3 sm	~15.5 min	~23.1%
VFR	Ceiling >3,000 ft · visibility >5 sm	~14.8 min	~22.7%
MVFR	Ceiling 1,000–3,000 ft · visibility 3–5 sm	~13.2 min	~21.5%

BI4 and BI7: Carrier Performance Benchmarking

There is a 2.7-fold difference in average departure delay between the best-performing carrier (Hawaiian Airlines at 7.9 minutes) and the worst (Comair at 21.2 minutes). A passenger flying Frontier Airlines is 76% more likely to experience a departure delay than one flying Hawaiian Airlines on the same day. Hawaiian Airlines' superior performance reflects its point-to-point island route structure, which minimises hub complexity and network cascade effects. Table 13 provides the complete ranking for all 14 US domestic carriers.

Table 13*: Carrier Performance Ranking*

Rank	Code	Carrier	Avg Dep Delay	Delay Rate	Flights
1	HA	Hawaiian Airlines	7.9 min	14.7%	73,249
2	AS	Alaska Airlines	8.5 min	19.9%	277,522
3	MQ	Envoy Air	9.2 min	17.5%	323,108
4	YX	Republic Airways	10.2 min	17.5%	370,173
5	DL	Delta Air Lines	11.2 min	18.5%	1,099,597
6	UA	United Airlines	12.0 min	18.1%	854,695
7	WN	Southwest Airlines	12.6 min	23.4%	1,488,636
8	OO	SkyWest Airlines	13.3 min	18.1%	900,156
9	NK	Spirit Airlines	14.1 min	21.9%	200,028
10	G4	Allegiant Air	15.1 min	22.5%	139,595
11	B6	JetBlue Airways	17.5 min	24.9%	247,915
12	AA	American Airlines	19.3 min	24.3%	1,039,723
13	F9	Frontier Airlines	19.3 min	25.8%	212,817
14	OH	Comair	21.2 min	24.3%	262,684

BI5: Delay Cause Attribution

The most important finding of the platform concerns the attribution of delay minutes by FAA cause category. late aircraft cascade account for approximately 39% of all delay minutes at the worst-performing airports, with the Carrier operational failures contributing with 36%. Together these two airline-controllable causes account for 75% of all delay minutes, with weather responsible for only 4%. This finding directly contradicts the common perception among passengers that most delays are weather-driven (Figure 9).

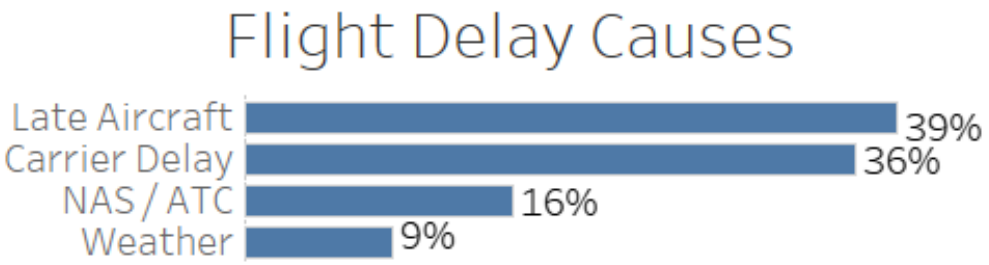

Figure 8: *Delay Cause Attribution*

BI6: Most Consistently Delayed Routes

The ten most delayed routes by average departure delay are characterised by three structural features: thin flight frequency, operational complexity, and geographic isolation. Allegiant Air (G4) appears four times in the top ten, confirming the structural vulnerability of ultra-low-cost point-to-point operations. The worst route, Palm Springs (PSP) to Charlotte (CLT) operated by American Airlines averages 184 minutes per departure over the study period, more than three hours of delay on average. Table 14 documents the ten worst routes.

Table 14:*Top 10 Most Delayed Routes by Average Departure Delay*

#	Origin	Dest	Carrier	Avg Delay	Delay Rate	Flights
1	PSP	CLT	AA	184.3 min	45.5%	11
2	MIA	RIC	YX	181.3 min	50.0%	12
3	OAK	MSO	G4	172.8 min	44.4%	18
4	CLT	ASE	OO	164.3 min	53.1%	32
5	STX	ORD	AA	140.1 min	30.8%	13
6	ORD	STX	AA	139.0 min	23.1%	13
7	STT	SJU	F9	135.7 min	64.3%	14
8	MYR	GRR	G4	128.7 min	46.7%	30
9	SJU	STT	F9	127.4 min	64.3%	14
10	CHS	LCK	G4	127.0 min	54.2%	24

8. Conclusions

8.1 Discussion

The Flight Delay Intelligence Platform successfully delivers a cloud-native data lake and data warehouse on Amazon Web Services, processing 7.4 million flight records and answering seven business intelligence questions through a fully automated nightly pipeline. Several significant architectural trade-offs were made during implementation, driven primarily by the constraints of the AWS. Table 15 documents each decision with its rationale.

Table 15: *Key Architectural Decisions and Trade-offs*

Decision	Choice Made	Alternative	Rationale
Query engine	Amazon Athena	Amazon Redshift	Redshift Serverless blocked by IAM policy; provisioned minimum cost USD 4,759/month. Athena integrates natively with Glue Catalog and delivers sub-2-second results on pre-aggregated gold Parquet.
Lambda scheduling	EventBridge classic rules	EventBridge Scheduler	Scheduler auto-creates IAM role — blocked in Sandbox. Classic rules reuse existing Lambda execution role.
Storage format	Parquet/Snappy	CSV	43× compression ratio (3.2 GB → 75 MB) and columnar reads reduce Athena scan cost by ~85%.
ETL framework	PySpark DataFrames	Glue DynamicFrames	DataFrames provide full schema control. DynamicFrames introduced type inference issues with mixed-type BTS columns.
Loading strategy	Full overwrite nightly	Incremental loading	BTS publishes retroactive corrections; incremental loading would miss these without complex change-detection logic.

8.2 Project Outcomes

The platform delivers four concrete outcomes. First, a fully automated end-to-end pipeline runs nightly from 00:00 to 01:30 Zurich time without manual intervention. Second, 7,399,922 BTS records are processed and compressed 43-fold from 3.2 GB CSV to 75 MB Parquet, enabling sub-two-second Athena queries. Third, six pre-aggregated gold tables provide documented, stable query interfaces for all seven BI questions. Fourth, the platform successfully delivers a fully operational data warehouse and visualisation layer built on Amazon Athena and Tableau, answering all seven business intelligence questions defined in Section 1.4.

The seven BI questions produced the following key findings. Departure delays follow a clear diurnal pattern: early morning flights carry low delay risk while flights after 7pm flights carry high delay risk in order of hours (BI1). The Nantucket Memorial to Chicago O'Hare route (ACK→ORD) is the most delayed in the network at 93 minutes average (BI2). IFR weather conditions produce a 23.1% delay rate compared to 22.7% under clear VFR conditions (BI3). Hawaiian Airlines is the best performing carrier at 7.9 minutes average departure delay, while Comair is the worst at 21.2 minutes (BI4 and BI7). Carrier operations are responsible for 65% of all delay minutes, confirming that most delays are airline-controllable rather than weather-driven (BI5). The PSP to CLT route averages 184 minutes per departure — the highest in the dataset (BI6).

8.3 Future Work

Five enhancements would materially extend the platform. Expanding METAR coverage from 20 to 500+ airports through the full ASOS/AWOS station network would dramatically improve the weather correlation analysis. Training a gradient boosting classifier on the gold-layer features (carrier, route, hour, and flight category) would extend the platform from historical analysis to predictive delay forecasting. Implementing S3 Intelligent Tiering on the bronze zone would automatically move older JSON snapshots to cheaper storage classes after 90 days, reducing the long-term storage cost of the immutable audit trail.

8.4 Generative AI Declaration

Generative AI tools were used selectively during this project as a productivity aid. LLMs were consulted for syntax debugging of Python and PySpark code, for checking SQL query logic, and for proofreading and improving the clarity of written sections.

9. Bibliography

Bureau of Transportation Statistics (BTS). (2025). On-Time Performance Data. United States Department of Transportation. https://www.transtats.bts.gov/

Databricks. (2025). What is the Medallion Lakehouse Architecture? https://docs.databricks.com/aws/en/lakehouse/medallion

Federal Aviation Administration (FAA). (2024). Air Traffic by the Numbers. https://www.faa.gov/air_traffic/by_the_numbers

Inmon, W. H., Strauss, D., & Neushloss, G. (2016). Data Lake Architecture: Designing the Data Lake and Avoiding the Garbage Dump. Technics Publications.

Kimball, R., & Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd ed.). Wiley.

Melnik, S., Gubarev, A., Long, J. J., Romer, G., Shivakumar, S., Tolton, M., & Vassilakis, T. (2010). Dremel: Interactive analysis of web-scale datasets. Proceedings of the VLDB Endowment.

NOAA Aviation Weather Center. (2025). METAR Data API. https://aviationweather.gov/api/data/metar

Amazon Web Services. (2024). Amazon Athena Documentation. https://docs.aws.amazon.com/athena/

Amazon Web Services. (2024). AWS Glue Developer Guide. https://docs.aws.amazon.com/glue/

AviationStack. (2025). AviationStack REST API Documentation. https://aviationstack.com/documentation

Appendix A — Athena SQL View Definitions

```
-- BI1: Airport hourly delay
CREATE OR REPLACE VIEW vw_airport_hourly AS
SELECT Origin AS airport, dep_hour, month, avg_delay, total_flights, delay_rate
FROM airport_hourly_delay WHERE total_flights >= 10;

-- BI2, BI6: Route delay summary
CREATE OR REPLACE VIEW vw_route_summary AS
SELECT OP_CARRIER AS carrier, Origin AS origin_airport, Dest AS dest_airport,
       avg_dep_delay, total_flights, delay_rate,
       avg_carrier_delay, avg_weather_delay, avg_nas_delay,
avg_late_aircraft_delay
FROM route_delay_summary WHERE total_flights >= 10;

-- BI3: Weather vs delay
CREATE OR REPLACE VIEW vw_weather_delay AS
SELECT Origin AS airport, OP_CARRIER AS carrier, visibility_sm,
       flight_category, DEP_DELAY AS dep_delay, is_delayed, wind_kt
FROM flight_weather_joined WHERE DEP_DELAY IS NOT NULL;

-- BI4, BI7: Carrier performance
CREATE OR REPLACE VIEW vw_carrier_perf AS
SELECT OP_CARRIER AS carrier, avg_dep_delay, avg_arr_delay,
       total_flights, delay_rate, total_cancelled
FROM carrier_performance ORDER BY delay_rate;

-- BI5: Delay cause breakdown
CREATE OR REPLACE VIEW vw_delay_causes AS
SELECT Origin AS airport, OP_CARRIER AS carrier, avg_carrier_delay,
       avg_weather_delay, avg_nas_delay, avg_late_aircraft_delay,
total_delayed_flights
FROM delay_cause_breakdown WHERE total_delayed_flights >= 5;

-- Live AviationStack
CREATE OR REPLACE VIEW vw_aviationstack AS
SELECT carrier, origin, dest, total_flights, avg_dep_delay, avg_arr_delay,
       delayed_flights, aircraft_type,
       CAST(delayed_flights AS DOUBLE) / total_flights AS delay_rate
FROM aviationstack_summary WHERE total_flights >= 2;
```

Appendix B — Business Intelligence Export Queries

```
-- BI1 → bi1_airport_hourly.csv
SELECT dep_hour,
   AVG(avg_delay) AS avg_delay,
   SUM(total_flights) AS    total_flights
FROM airport_hourly_delay
WHERE total_flights >= 10
GROUP BY dep_hour
ORDER BY dep_hour ASC;

-- BI2 → bi2_route_summary.csv
SELECT OP_CARRIER, Origin, Dest, avg_dep_delay,
       total_flights, delay_rate, avg_carrier_delay,
       avg_weather_delay, avg_nas_delay, avg_late_aircraft_delay
FROM route_delay_summary
ORDER BY delay_rate DESC
LIMIT 500;

-- BI3 → bi3_weather_delay.csv
SELECT flight_category,
       COUNT(*) AS total_flights,
       AVG(DEP_DELAY) AS avg_delay,
       SUM(CASE WHEN is_delayed THEN 1 ELSE 0 END)*100.0/COUNT(*) AS
delay_rate_pct
FROM flight_weather_joined
WHERE DEP_DELAY IS NOT NULL
AND flight_category IS NOT NULL
AND DEP_DELAY BETWEEN -60 AND 500
GROUP BY flight_category
ORDER BY avg_delay DESC;

-- BI4 and BI7 → bi4_carrier_performance.csv
SELECT OP_CARRIER, avg_dep_delay, avg_arr_delay,
       total_flights, delay_rate, total_cancelled
FROM carrier_performance
ORDER BY delay_rate ASC;

-- BI5 → bi5_delay_causes.csv
SELECT
    'Carrier Delay' AS cause,
    AVG(avg_carrier_delay) AS avg_minutes,
    AVG(avg_carrier_delay) / (AVG(avg_carrier_delay) + AVG(avg_weather_delay)
    + AVG(avg_nas_delay) + AVG(avg_late_aircraft_delay)) * 100 AS pct
FROM delay_cause_breakdown
UNION ALL
SELECT
    'Late Aircraft' AS cause,
    AVG(avg_late_aircraft_delay) AS avg_minutes,
    AVG(avg_late_aircraft_delay) / (AVG(avg_carrier_delay) +
AVG(avg_weather_delay)
    + AVG(avg_nas_delay) + AVG(avg_late_aircraft_delay)) * 100 AS pct
FROM delay_cause_breakdown
UNION ALL
SELECT
    'NAS / ATC' AS cause,
    AVG(avg_nas_delay) AS avg_minutes,
    AVG(avg_nas_delay) / (AVG(avg_carrier_delay) + AVG(avg_weather_delay)
    + AVG(avg_nas_delay) + AVG(avg_late_aircraft_delay)) * 100 AS pct
FROM delay_cause_breakdown
UNION ALL
SELECT
    'Weather' AS cause,
    AVG(avg_weather_delay) AS avg_minutes,
    AVG(avg_weather_delay) / (AVG(avg_carrier_delay) + AVG(avg_weather_delay)
```

```
    + AVG(avg_nas_delay) + AVG(avg_late_aircraft_delay)) * 100 AS pct
FROM delay_cause_breakdown
ORDER BY pct DESC;

-- BI6 → bi6_top_routes.csv
SELECT OP_CARRIER, Origin, Dest,
       avg_dep_delay, delay_rate, total_flights
FROM route_delay_summary
ORDER BY avg_dep_delay DESC
LIMIT 30;
```